Oracle HR Schema

Subquery Analytics & Business Reporting

Database Systems — Laboratory Report

Oracle SQL | HR Schema | Subquery Techniques

Objective

Develop analytical reports using subqueries in the Oracle HR Schema, demonstrating understanding of business requirements, query execution, and different subquery techniques including scalar, multi-row, correlated, inline view, and nested subqueries. Oracle SQL syntax is used throughout. Each query is explained with its business purpose and subquery classification.

Subquery Types Reference

Subquery Type	Where Used	Returns	Key Characteristic
Scalar	WHERE / SELECT / HAVING	1 row, 1 col	Compared like a single literal value
Multi-row	WHERE IN / NOT IN / ANY / ALL	Many rows	Set membership or exclusion test
Correlated	WHERE (references outer query)	Varies	Re-executes once per outer row
Inline View	FROM clause	Virtual table	Pre-aggregates data before outer filter
Nested	Inside another subquery	Varies	Inner subquery executes first

Contents

Section 1 — Salary Analysis Using Subqueries

1.1 Employees earning more than the company average salary

Type: Scalar

Logic: The inner query calculates AVG(salary) once across all employees — a single value. The outer WHERE compares every employee salary against that one number. Because the subquery always returns exactly one row and one column, it is a scalar subquery.

SQL:

```
SELECT employee_id, first_name, last_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

1.2 Employees earning less than the company highest salary

Type: Scalar

Logic: MAX(salary) is computed once by the inner query and used as a ceiling value. The outer query returns everyone whose salary is strictly below that ceiling — which excludes only the employee(s) holding the absolute maximum.

SQL:

```
SELECT employee_id, first_name, salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

1.3 Departments whose average salary exceeds the company average

Type: Scalar in HAVING

Logic: GROUP BY aggregates employees per department. HAVING filters those groups using a scalar subquery that returns the company-wide AVG. This compares a department-level aggregate against a company-level aggregate in one query.

SQL:

```
SELECT department_id, AVG(salary) AS dept_avg
FROM employees
GROUP BY department_id
HAVING AVG(salary) > (SELECT AVG(salary) FROM employees);
```

1.4 Employees earning the highest salary within their department

Type: Correlated

Logic: For each row the outer query evaluates, Oracle re-executes the inner query passing that row's department_id to find MAX(salary) for that specific department. The employee's salary is then compared to their own department's maximum — row by row. This row-by-row re-execution is the defining feature of a correlated subquery.

SQL:

```
SELECT employee_id, first_name, department_id, salary
FROM employees e
WHERE salary = (SELECT MAX(salary)
FROM employees
WHERE department_id = e.department_id);
```

Section 2 — Department & Location Analysis

2.1 Employees working in departments located in the United States

Type: Multi-row (IN) — Nested

Logic: Two-level lookup without JOIN: the inner subquery finds location_ids in the US (country_id = 'US'), the middle subquery maps those to department_ids, and the outer WHERE uses IN to filter employees in those departments.

SQL:

```
SELECT employee_id, first_name, department_id
FROM employees
WHERE department_id IN (
SELECT department_id
FROM departments
WHERE location_id IN (
SELECT location_id
FROM locations
WHERE country_id = 'US'));
```

2.2 Departments that currently contain employees

Type: Multi-row (IN)

Logic: DISTINCT department_id from the employees table gives every department that has at least one employee. IN matches each row in the departments table against that set.

SQL:

```
SELECT department_id, department_name
FROM departments
WHERE department_id IN (
SELECT DISTINCT department_id FROM employees);
```

2.3 Departments that currently have no employees

Type: Multi-row (NOT IN)

Logic: NOT IN is the logical inverse — it excludes every department_id that appears in the employees table. The IS NOT NULL guard prevents the NULL trap: if any department_id in employees is NULL, NOT IN returns zero rows without it.

SQL:

```
SELECT department_id, department_name
FROM departments
WHERE department_id NOT IN (
SELECT department_id
FROM employees
WHERE department_id IS NOT NULL);
```

2.4 Employees in departments located in the Europe region (nested subqueries)

Type: Nested (5 levels)

Logic: Five nesting levels flow from innermost to outermost: (1) find Europe region_id, (2) find country_ids in that region, (3) find location_ids in those countries, (4) find department_ids at those locations, (5) filter employees. Oracle always evaluates the innermost subquery first.

SQL:

```
SELECT employee_id, first_name
FROM employees
WHERE department_id IN (
SELECT department_id FROM departments
WHERE location_id IN (
SELECT location_id FROM locations
WHERE country_id IN (
SELECT country_id FROM countries
WHERE region_id = (
SELECT region_id FROM regions
WHERE region_name = 'Europe'))));
```

Section 3 — Manager & Hierarchy Analysis

3.1 Employees earning more than their manager

Type: Correlated

Logic: The outer query supplies the manager_id of each employee. The correlated subquery looks up the salary of exactly that manager (WHERE employee_id = e.manager_id). If the employee's salary exceeds the manager's, the row is returned. A self-join implemented as a correlated subquery.

SQL:

```
SELECT e.employee_id, e.first_name, e.salary
FROM employees e
WHERE e.salary > (SELECT salary
FROM employees
WHERE employee_id = e.manager_id);
```

3.2 Managers supervising at least one employee earning more than 10,000

Type: Multi-row (IN)

Logic: The subquery collects all distinct manager_ids where any subordinate earns above 10,000. DISTINCT prevents duplicates when one manager has multiple qualifying subordinates. The outer query retrieves those managers by matching employee_id.

SQL:

```
SELECT employee_id, first_name, salary
FROM employees
WHERE employee_id IN (
SELECT DISTINCT manager_id
FROM employees
WHERE salary > 10000);
```

3.3 Employees whose manager works in the same department

Type: Correlated

Logic: For each employee the correlated subquery fetches the department_id of that employee's manager. The WHERE clause checks whether the manager's department equals the employee's own department_id — a row-by-row comparison.

SQL:

```
SELECT e.employee_id, e.first_name, e.department_id
FROM employees e
WHERE e.department_id = (SELECT department_id
FROM employees
WHERE employee_id = e.manager_id);
```

3.4 Employees whose manager works in a different department

Type: Correlated

Logic: The != operator inverts query 3.3 — identifying cross-department reporting relationships. Useful for matrix or project-based org structures where an employee reports to a manager outside their own department.

SQL:

```
SELECT e.employee_id, e.first_name, e.department_id
FROM employees e
WHERE e.department_id != (SELECT department_id
FROM employees
WHERE employee_id = e.manager_id);
```

Section 4 — Inline View & Query Execution

4.1 Filtering employees earning more than 10,000

Type: Inline View

Logic: The subquery in the FROM clause creates a named virtual table called high_earners. The outer query selects from it exactly like a real table. Inline views are especially useful when the derived dataset needs sorting, aliasing, or additional filtering before the outer query processes it.

SQL:

```
SELECT *
FROM (SELECT employee_id, first_name, salary
FROM employees
WHERE salary > 10000) high_earners;
```

4.2 Displaying departments with average salary greater than 9,000

Type: Inline View

Logic: The inline view pre-computes AVG(salary) per department and exposes it as the column avg_sal. The outer WHERE clause filters on it directly — which is not possible with a plain WHERE (aggregate functions cannot appear there). This is a key advantage of the inline view pattern.

SQL:

```
SELECT department_id, avg_sal
FROM (SELECT department_id, AVG(salary) avg_sal
FROM employees
GROUP BY department_id)
WHERE avg_sal > 9000;
```

4.3 Displaying departments with more than five employees

Type: Inline View

Logic: COUNT(*) per department is computed inside the inline view and aliased as emp_count. The outer query filters where emp_count > 5. Equivalent to HAVING COUNT(*) > 5 but demonstrates how inline views make aggregation results composable for outer queries.

SQL:

```
SELECT department_id, emp_count
FROM (SELECT department_id, COUNT(*) emp_count
FROM employees
GROUP BY department_id)
WHERE emp_count > 5;
```

Section 5 — Advanced Subquery Logic

5.1 Employees earning above their department average salary

Type: Correlated

Logic: e.department_id from the outer row is passed into the inner query to compute AVG(salary) for that specific department only. The outer employee's salary is compared to that result. Runs once per employee row.

SQL:

```
SELECT e.employee_id, e.first_name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (SELECT AVG(salary)
FROM employees
WHERE department_id = e.department_id);
```

5.2 Employees earning above the average salary for their job role

Type: Correlated

Logic: Same correlated pattern as 5.1 but the correlation key is job_id instead of department_id. Identifies employees who earn above the average for their specific job title across the whole company — useful for pay-equity analysis.

SQL:

```
SELECT e.employee_id, e.first_name, e.job_id, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary)
FROM employees
WHERE job_id = e.job_id);
```

5.3 Departments where all employees earn more than 5,000

Type: Multi-row (NOT IN)

Logic: NOT IN logic: exclude every department that contains at least one employee earning <= 5,000. The remaining departments must be those where every single employee earns above 5,000. Equivalent to HAVING MIN(salary) > 5000.

SQL:

```
SELECT DISTINCT department_id
FROM employees
WHERE department_id NOT IN (
SELECT department_id
FROM employees
WHERE salary <= 5000);
```

5.4 Employees who are the only employee in their department

Type: Correlated

Logic: The correlated subquery counts how many employees share the same department_id as the current outer row. A COUNT of exactly 1 means that employee is the sole person in their department — useful for identifying understaffed or isolated units.

SQL:

```
SELECT employee_id, first_name, department_id
FROM employees e
WHERE 1 = (SELECT COUNT(*)
FROM employees
WHERE department_id = e.department_id);
```

Section 6 — Executive HR Analytics Challenge

6.1 Employees earning above the company average salary

Type: Scalar

Logic: Scalar subquery returns one AVG value computed once. The outer WHERE filters employees whose salary exceeds it. This is the baseline executive salary benchmark.

SQL:

```
SELECT employee_id, first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

6.2 Departments with more employees than Department 50

Type: Inline View + Scalar

Logic: The inline view counts employees per department. A scalar subquery counts employees specifically in department 50. The outer query compares each count to that benchmark. Demonstrates combining two subquery types in one statement.

SQL:

```
SELECT department_id, emp_count
FROM (SELECT department_id, COUNT(*) emp_count
FROM employees GROUP BY department_id)
WHERE emp_count > (SELECT COUNT(*)
FROM employees
WHERE department_id = 50);
```

6.3 Employees working in the department with the highest average salary

Type: Nested (3 levels)

Logic: (1) AVG per department, (2) MAX of those averages — the top department average, (3) which department_id holds that peak average, (4) employees in that department. Oracle executes from innermost outward.

SQL:

```
SELECT employee_id, first_name, department_id
FROM employees
WHERE department_id = (
SELECT department_id
FROM employees
GROUP BY department_id
HAVING AVG(salary) = (
SELECT MAX(AVG(salary))
FROM employees
GROUP BY department_id));
```

6.4 Employees earning the second-highest salary

Type: Nested Scalar

Logic: Inner subquery finds the absolute MAX salary. Middle subquery finds the MAX salary strictly below that maximum — which is by definition the second-highest value. Outer query returns all employees whose salary equals that value.

SQL:

```
SELECT employee_id, first_name, salary
FROM employees
WHERE salary = (SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary)
FROM employees));
```

6.5 Managers whose employees' average salary exceeds 10,000

Type: GROUP BY + HAVING

Logic: GROUP BY manager_id aggregates all direct reports per manager. HAVING AVG(salary) > 10000 keeps only managers whose team average exceeds the threshold. A team performance metric used for compensation benchmarking.

SQL:

```
SELECT manager_id
FROM employees
WHERE manager_id IS NOT NULL
GROUP BY manager_id
HAVING AVG(salary) > 10000;
```

Key Reminders

- Use Oracle SQL syntax only — standard ANSI may differ in edge cases.
- Correlated subqueries re-execute once per outer row — can be slow on large tables; consider RANK() or DENSE_RANK() analytic functions for ranking tasks.
- NOT IN with NULLs returns zero rows — always include IS NOT NULL in the subquery.
- Inline views expose aggregate results as plain columns usable in the outer WHERE clause.
- Nested subqueries always execute from innermost to outermost.
- The second-highest salary pattern (Section 6, Query 6.4) is a classic SQL interview question.
- The scalar subquery in HAVING (Section 1, Query 1.3) compares two levels of aggregation in one statement.